



Tuxhydro

Análisis de la influencia de los índices oceánicos en los aportes hídricos en los embalses en Colombia.

Tuxito Lovers

Universidad Jorge Tadeo Lozano

Alvarado Becerra Ludwig

www.ludwigalvarado.me

Jiménez Quintero Nicolle Tatiana

tatiana.jima@gmail.com

Martínez Guerrero Juan José

info@cuspidе.club

Ojeda Mejía María Sofía

sofiaojedamejia2004@gmail.com

Resumen

Tuxilo![2]

Índice general

Resumen	i
Índice general	ii
Índice de figuras	iii
1 Descripción general de la solución	1
2 Análisis técnico y metodología	2
3 Uso de datos y fuentes	3
4 Herramientas, requerimientos de software y arquitectura	6
5 Código y replicabilidad	7
6 Resultados y visualización	8
7 Guía de despliegue e instalación operativa	9
A Apéndice A	10
B Acrónimos	11
Bibliografía	13

Índice de figuras

3.1	Mapa hecho con Leaflet con datos base de OpenStreetMap y capas extra de embalses de Colombia.	5
-----	---	---

CAPÍTULO 1

Descripción general de la solución

CAPÍTULO 2

Análisis técnico y metodología

CAPÍTULO 3

Uso de datos y fuentes

Se identificaron y emplearon datos correspondientes a los polígonos y áreas de los embalses, los cuales fueron obtenidos a partir de Colombia en Mapas y el Instituto Geográfico Agustín Codazzi (IGAC) (<https://www.colombiaenmapas.gov.co/?u=0&t=39&servicio=1468>)[3]. Estos datos se encuentran disponibles dentro del repositorio del proyecto, específicamente en la ruta <https://github.com/GNUTADEO/Tuxilo/tree/main/data/geodata>, donde se almacenan los distintos archivos resultantes de los procesos de consulta y tratamiento geoespacial.

El archivo `embalses.shp` fue transformado en un *script* SQL con el objetivo de permitir la correcta carga de la información en la base de datos espacial PostGIS. Este proceso se realizó mediante el uso del comando `shp2pgsql`, junto con las instrucciones mostradas en el listado 3.1. Como resultado, se generó el archivo `init-embalses-geom-shp2pgsql-2.sql`, el cual contiene las sentencias necesarias para la creación y carga de la tabla `embalses_geom` dentro de la base de datos PostGIS.

```
1 shp2pgsql -I -W UTF-8 -g geom -k embalses.shp >
   init-embalses-geom-shp2pgsql-2.sql
```

Listing 3.1: Conversión de un archivo `.shp` a un script SQL

Una vez cargadas las geometrías de los archivos `.shp`, se procedió a calcular el centroide de cada polígono con el fin de obtener sus coordenadas representativas. A partir de estos centroides, se pobló la tabla `embalses`, almacenando las coordenadas de latitud y longitud correspondientes. El *script* completo utilizado para la creación y población de ambas tablas se encuentra en el archivo `init-db.sql`, ubicado en el directorio `scripts/`.

Estos dos conjuntos de datos son expuestos a través de una API, lo que permite su consulta y reutilización por parte de distintos módulos internos del sistema. La información general de los embalses puede consultarse mediante el *endpoint* `/public/embalses/`, mientras que los polígonos asociados están disponibles a través del *endpoint* `/public/embalses/geojson`. Ambos *endpoints* se encuentran accesibles a través del puerto 8000 del `localhost`, y su implementación se muestra en el listado 3.2.

La capa base de datos geográficos fue obtenida a partir de la base de datos espacial de OpenStreetMap [1], haciendo uso de la información conforme a los términos establecidos por la Licencia Abierta de Bases de Datos (ODbL, *Open Database License*) [5]. Los elementos representados sobre el mapa fueron incorporados como capas vectoriales superpuestas, particularmente en el caso de los marcadores de ubicación.

Adicionalmente, se generó una capa en formato GeoJSON [6], obtenida mediante la consulta al *endpoint* `/public/embalses/geojson`, lo que permite la representación visual de los polígonos correspondientes a los embalses. La integración y visualización de los datos cartográficos se realizó con ayuda de la librería Leaflet [4] para JS, obteniendo como resultado una interfaz similar a la presentada en la figura 3.1.

```
1  router = APIRouter(tags=["Embalses"], prefix="/embalses")
2
3
4  @router.get(
5  "/",
6  operation_id="get_all_embalses",
7  )
8  async def get_all_embalses(
9  db: AsyncSession = Depends(get_db),
10 ):
11     """Obtiene todos los embalses con sus coordenadas"""
12     query = text("""
13     SELECT
14     id,
15     nombre,
16     latitud,
17     longitud
18     FROM embalses
19     WHERE latitud IS NOT NULL
20     AND longitud IS NOT NULL
21     AND nombre IS NOT NULL
22     """)
23     result = await db.execute(query)
24     rows = result.fetchall()
25
26     embalses = [
27     {
28         "id": row[0],
29         "nombre": row[1],
30         "latitud": float(row[2]) if row[2] else None,
31         "longitud": float(row[3]) if row[3] else None,
32     }
33     for row in rows
34     ]
35
36     return {"embalses": embalses}
37
38 @router.get("/geojson")
39 async def get_embalses_geojson(db: AsyncSession = Depends(get_db)):
40     query = text("""
41         SELECT jsonb_build_object(
```

```

42     'type', 'FeatureCollection',
43     'features', jsonb_agg(feature)
44 )
45 FROM (
46     SELECT jsonb_build_object(
47         'type', 'Feature',
48         'id', "GLOBALID",
49         'geometry', ST_AsGeoJSON(ST_Transform(ST_SetSRID(geom,
50             9377), 4326))::jsonb,
51         'properties', jsonb_build_object(
52             'nombre', "NOMBRE_GEO",
53             'proyecto', "PROYECTO",
54             'area_km2', ROUND(("SHAPE_Area"/1000000)::numeric,
55                 2)
56         ) AS feature
57     FROM embalses_geom WHERE "NOMBRE_GEO" IS NOT NULL
58 ) features;
59 result = await db.execute(query)
60 return result.scalar()

```

Listing 3.2: Archivo App/Core/src/routers/embalses.py

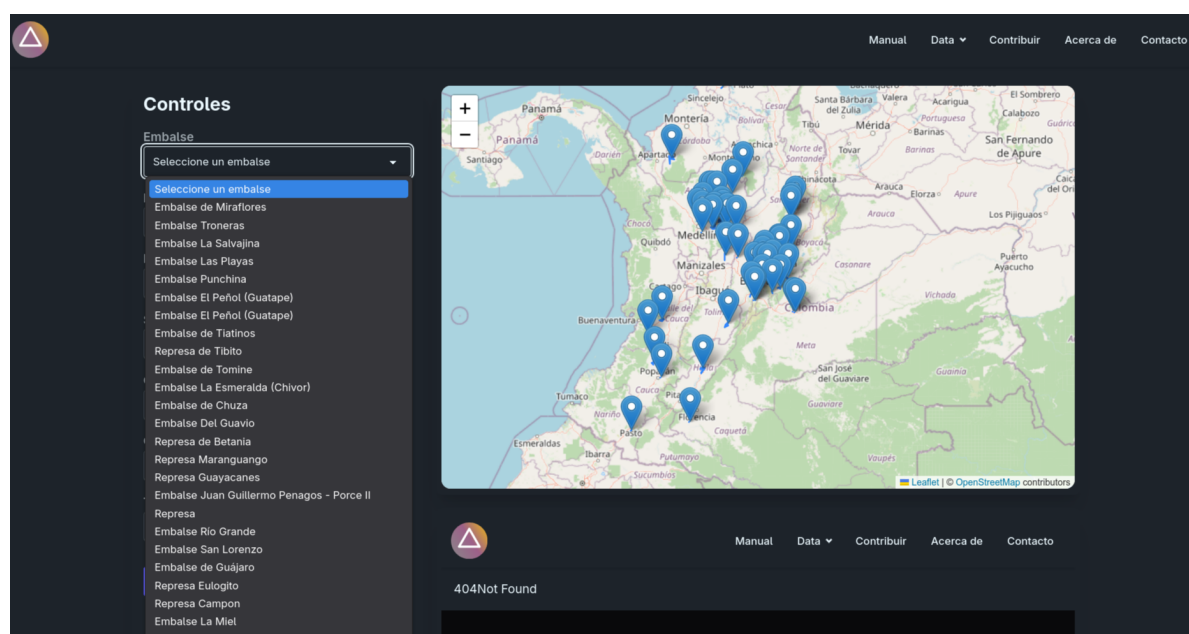


Figura 3.1: Mapa hecho con Leaflet con datos base de OpenStreetMap y capas extra de embalses de Colombia.

CAPÍTULO 4

Herramientas, requerimientos de software y arquitectura

CAPÍTULO 5

Código y replicabilidad

CAPÍTULO 6

Resultados y visualización

CAPÍTULO 7

Guía de despliegue e instalación operativa

APÉNDICE A

Apéndice A

APÉNDICE B

Acrónimos

Siglas

API Application Programming Interface

IGAC Instituto Geográfico Agustín Codazzi

JS JavaScript

ODbL Open Database License

Bibliografía

- [1] *About OpenStreetMap*. Official OpenStreetMap “About” page. OpenStreetMap Foundation. 2025. URL: <https://www.openstreetmap.org/about> (visitado 14 de diciembre de 2025).
- [2] GNUTADEO. *Tuxilo: Análisis Hidroeléctrico y Climático*. GitHub repository. 2025. URL: <https://github.com/GNUTADEO/Tuxilo/> (visitado 10 de diciembre de 2025).
- [3] Instituto Geográfico Agustín Codazzi (IGAC). *Portal de información geoespacial – Colombia en Mapas*. Accedido: 20 de diciembre de 2025. IGAC. 2025. URL: <https://www.colombiainmapas.gov.co/?u=0&t=39&servicio=1468> (visitado 20 de diciembre de 2025).
- [4] Leaflet contributors. *Leaflet — a JavaScript library for interactive maps*. Accedido el 16 de diciembre de 2025. Leaflet. 2025. URL: <https://leafletjs.com/> (visitado 16 de diciembre de 2025).
- [5] *Open Data Commons Open Database License (ODbL)*. Licencia de base de datos abierta con condiciones de atribución y “share-alike” para datos y bases de datos. Open Data Commons / Open Knowledge Foundation. 2025. URL: <https://opendatacommons.org/licenses/odbl/> (visitado 16 de diciembre de 2025).
- [6] Wikipedia contributors. *GeoJSON*. Accedido el 16 de diciembre de 2025. Wikipedia. 2025. URL: <https://en.wikipedia.org/wiki/GeoJSON> (visitado 16 de diciembre de 2025).